

Fronthaul Load Management in Open RAN

Part A: shrinking the load with matrix-decomposition compression

Part B: sharing one fronthaul link between cells with forecast-aware budgets

B.Tech. Project — mid-term evaluation

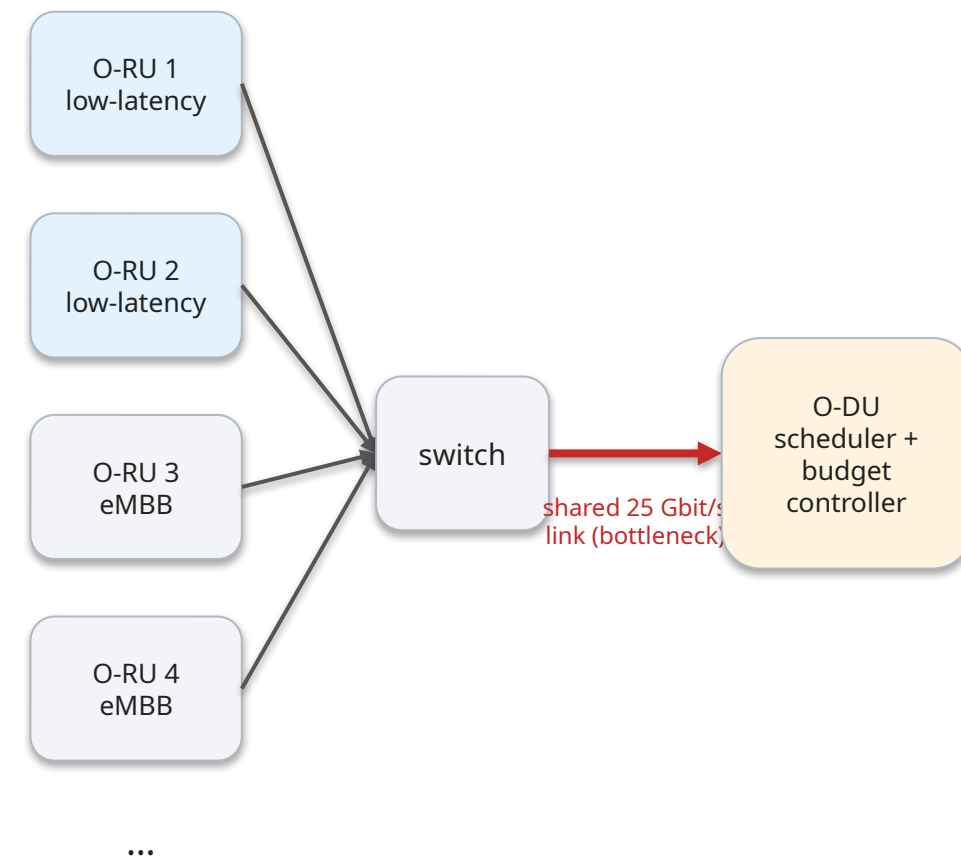
Student: <name> | Supervisor: <name> | Department of <dept>

What exists today: compression study + matrix-maths benchmark (Part A); working simulator, 7 controllers, trained forecaster, 300+ experiment runs, tests and this deck (Part B). All generated from the repository.

The problem: the fronthaul carries a lot of data

1 Motivation

- In the 7-2x split, the radio unit (O-RU) sends radio samples (IQ data) to the processing unit (O-DU)
 - Every scheduled resource block costs about 3,387 bits on the link (9-bit compression, 8% packet overhead)
 - One 100 MHz cell with 4 antenna layers can need up to 7.4 Gbit/s
- Several cells share one link that is smaller than their total peak — on purpose (statistical multiplexing)
 - Our setup: 8 cells on a 25 Gbit/s link; total peak = $2.4 \times$ the link
- When bursts coincide, some data must wait or be dropped. Low-latency cells can wait ~2 ms at most
- **Two ways to manage the load:**
 - A. Make the data smaller — compression (matrix decomposition, quantisation)
 - B. Share the link well — decide every few milliseconds how much each cell may send



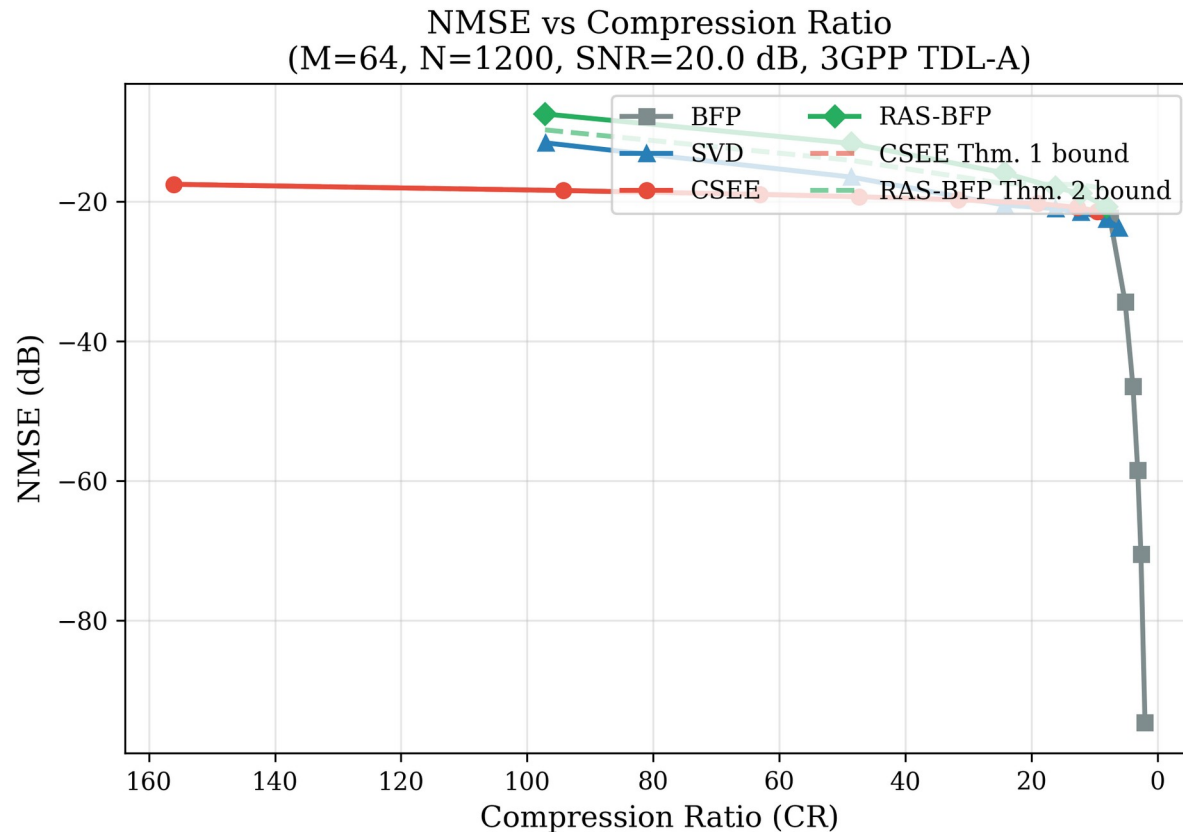
Part A — make the data smaller (earlier work)

- Compression of the IQ matrix Y (64 antennas $\times$ 1200 subcarriers) per slot
- Baselines: O-RAN block floating point (BFP), truncated SVD
- Studied: RAS-BFP (random sketch + QR $\rightarrow$ low rank), CSEE (keep the strongest delay taps)
- ACAFS: pick the functional split per user from the channel rank
- Benchmark of the matrix maths behind it: LU, QR, SVD, Newton-Schulz, learned inverters ($n = 10 \dots 500$)
- Output: 4 figure suites, benchmark CSV, report. Compression source package is not in this repo (see status)

Part B — share the link well (this term)

- Slot-level simulator of 8 cells on one 25 Gbit/s link, with queues, deadlines and priorities
- Every 10 ms a controller sets how many bits per slot each cell may send
- 5 reference controllers, from 'equal share' to 'knows the future'
- Proposed: forecast each cell's need as a range (quantiles), then split the link so that every cell is cut at the same weighted risk
- 200 test runs on 5 traffic scenarios, ablations, 14 unit tests
- Output: raw metrics, figures, this deck — all regenerated by scripts

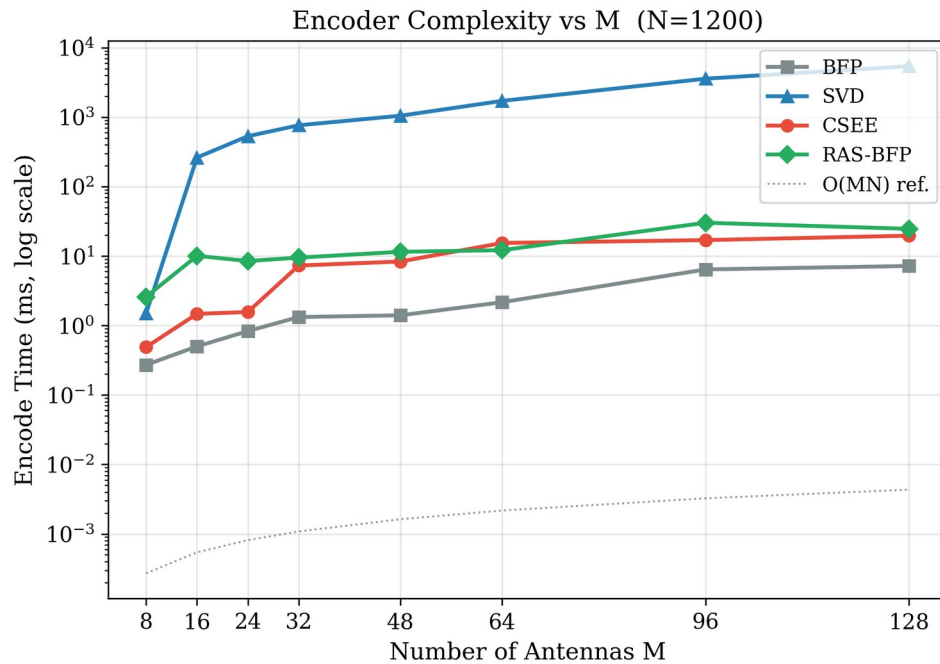
How they connect: compression decides how much load reaches the link (Part A lever). Allocation decides how the loss is shared when bursts coincide (Part B lever). Slide 15 shows both levers on the same traffic.



- Setup: $Y \in \mathbb{C}^{(64 \times 1200)}$ per slot, 3GPP TDL-A channel, SNR 20 dB (simulated)
- BFP (O-RAN standard): shared exponent + short mantissa. $\sim 4\times$ smaller, -46 dB error, ~ 3 ms
- Truncated SVD: keep the r strongest directions. $\sim 16\times$ at -21 dB, but 18-30 ms per slot
- RAS-BFP: random sketch $\Omega \cdot Y \rightarrow$ thin QR $\rightarrow Y \approx L \cdot Q^H$, then BFP. $\sim 16\times$ at -18 dB, ~ 7 ms (3-4 $\times$ faster than SVD)
- CSEE: IFFT along subcarriers $\rightarrow$ keep top-K delay taps $\rightarrow$ BFP. $\sim 13\times$ at -21 dB (~ 5 ms); $\sim 63\times$ at -19 dB with $K = 60$
- ACAFS: choose split 7.2x / 7.1 / 6 per user from the channel rank $\rightarrow$ in simulation 0 % saving below ~ 15 dB SNR, 55-75 % above (vs fixed split 6)
- **Status: figures and report exist (results/figures, docs/mid_evaluation_report.md); the src/ package is missing from the repo, so these numbers are quoted, not re-run**

Part A: how fast is the matrix maths? (measured on CPU)

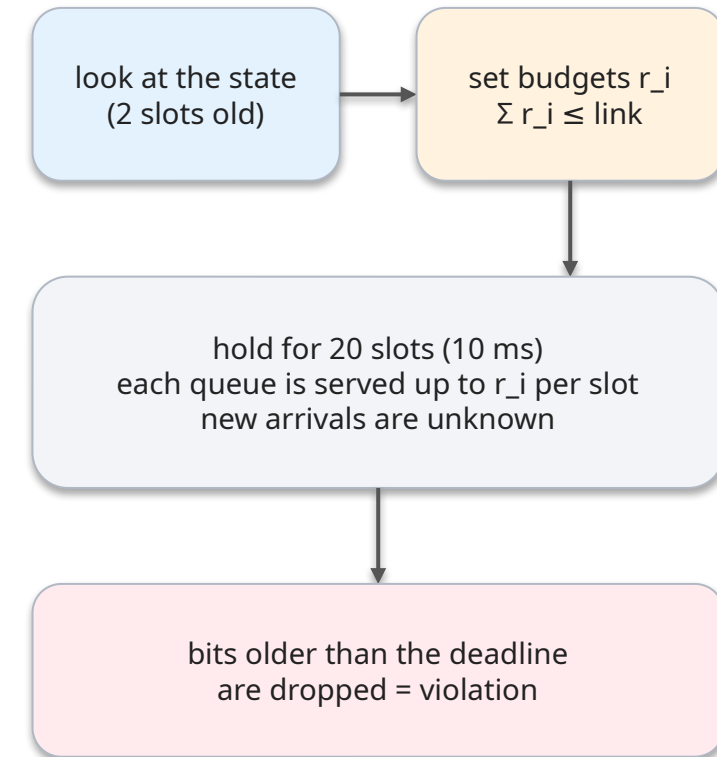
Part A



Method	n=100: median ms	n=100: rel. error	n=500: median ms	n=500: rel. error
LU via LAPACK (numpy.linalg.inv)	0.17	5e-08	18.00	5e-08
LU (scipy)	2.87	1e-07	56.29	2e-07
QR	3.94	3e-07	102.23	3e-07
SVD	2.76	1e-07	106.92	2e-07
Newton-Schulz iteration	1.01	2e-07	70.30	3e-07
Learned iteration (InverseNet-Ultra)	0.46	1e-06	33.79	1e-06
Direct neural net (InverseNet-MLP)	4.58	5e-01	n/a	n/a

- Encoder time grows fastest for SVD; sketch (RAS-BFP) and delay-domain (CSEE) encoders stay 10-100× below it
- Inversion benchmark (right, medians, 200 held-out matrices per size, well-conditioned): LU via LAPACK is fastest and exact
- A direct neural net that outputs the inverse does not work ($\approx 50\%$ error). A learned Newton-Schulz iteration does ($\approx 1e-6$ at $n = 100$) and has a fixed, predictable cost
- Lesson for slot deadlines (0.5 ms): $n \approx 100$ matrices fit per slot on a CPU; $n \approx 500$ do not → do them less often, incrementally, or on an accelerator
- Lesson for Part B: predictable compute time matters as much as average speed — the allocation rule below is a closed form plus one scalar search

- Time: slots of 0.5 ms. Data: bits. Link: bits per slot
- 8 cells: 3 low-latency (must be served within 4 slots = 2 ms, priority 3) and 5 eMBB (20 slots = 10 ms, priority 1)
- Fronthaul bits = user bits $\div$ spectral efficiency $\times$ bits per resource block — so fronthaul load $\propto$ user traffic
- Each cell has a first-in-first-out queue in the O-DU; bits older than the deadline are dropped and counted
- Link rule: budgets add up to at most 97 % of 25 Gbit/s (3 % kept for control traffic); no cell gets more than its own peak
- Every $T = 20$ slots (10 ms) the controller sets the budgets; they are fixed for the interval
- The controller sees the state 2 slots late; it never sees future arrivals (checked by a unit test)
- **Why decide in advance? A budget is a reservation held for 10 ms; the traffic in that window is not known yet**



Sources: O-RAN WG4 CUS-plane spec (7-2x, BFP, timing windows); Larsen et al., IEEE COMST 2019 (bit rate depends on load, ~8 % Ethernet overhead); Pérez et al. 2018/2019 (packet-switched aggregation).

- **A. The fronthaul itself (standards and surveys)**

- O-RAN WG4 CUS-plane spec (7-2x, BFP, timing windows) • Larsen et al., IEEE COMST 2019: bit rate varies with load
- Pérez et al., JOCN 2018 / IEEE Access 2019 (packet aggregation) • Wang & Zhou, IEEE Commun. Lett. 2017 (multiplexing gain)
- Lagén et al., IEEE Commun. Mag. 2022: several 7-2x cells share one link, compression is adjusted per slot — reactive, no forecast

- **B. Forecast first, then allocate capacity**

- Bega et al., DeepCog (INFOCOM 2019 / JSAC 2020), AZTEC (INFOCOM 2020): deep-learning forecasts for network slices
- Predictive bandwidth allocation for PON fronthaul (Mikaeil et al. 2018; Zhang et al. 2019); Kavehmadavani et al., TWC 2023
- Common pattern: one predicted number + a fixed safety margin; no per-cell deadlines

- **C. Forecasts with uncertainty**

- Cohen et al., IEEE WCL 2023 (conformal prediction for one URLLC stream) • Gibbs & Candès, NeurIPS 2021 (adaptive conformal)
- Kasuluru et al. 2024 [preprint]: probabilistic PRB-load forecasts; the percentile is picked globally

- **Gap: nobody turns a forecast RANGE of deadline-sensitive demand into per-cell budgets under one shared link, and checks it against strong reactive and point-forecast controllers**

- Decision every 10 ms: budgets $r_i \geq 0$, $\sum r_i \leq \text{link capacity}$, $r_i \leq \text{cell peak}$
- **Main score (fixed before tuning): priority-weighted deadline-violation ratio**
 - $= (3 \times \text{dropped low-latency bits} + \text{dropped eMBB bits}) / (3 \times \text{arrived low-latency bits} + \text{arrived eMBB bits})$
 - Also reported: violations per class, link utilisation, queueing delay, fairness (Jain index), decision time
- **'Demand' of a cell = deadline-feasible rate r^* : the smallest constant rate that serves the next arrivals with no bit waiting longer than its deadline D**
 - $r^* = \max \text{ over time windows of } (\text{bits in window}) / (\text{window length} + D)$ — a standard network-calculus bound
 - For eMBB ($D = 10$ ms) $r^* \approx \text{average rate}$. For low-latency cells ($D = 2$ ms) $r^* \approx \text{peak 2-ms burst rate}$ → hard to predict
 - Budget needed = rate to clear the current backlog in time + r^* of the arrivals we have not seen yet
- Why a single predicted number is not enough: r^* is skewed (usually low, sometimes very high). The median under-reserves; a fixed margin wastes capacity on every cell

Reference controllers (all implemented and tuned on validation)

Part B — baselines

Controller	How it sets budgets	What it uses	Tuned
Static equal share	link ÷ 8 per cell (spare capacity redistributed)	nothing	—
Reactive proportional	backlog / T + running average of demand; proportional split; spare redistributed	history	margin, window
Deadline-aware reactive (strong)	backlog-clearing rate + r^* of the last interval; priority weights; water-fill	history, deadlines, priorities	margin, #intervals
Point forecast + water-fill (ablation)	same as above, but r^* comes from the forecast MEDIAN	+ trained model	margin
Oracle water-fill (reference)	true r^* of the next interval, same water-fill. Perfect point forecast; not a bound on other rules	future arrivals	—

- **Tuning on validation seeds 2000-2003 (scenarios moderate/high); test seeds were never used:**

- reactive_prop → {'margin': 0.0, 'ewma_window': 20} | queue_aware → {'margin': 0.0, 'num_horizons': 1} | point_forecast → {'margin': 0.0} | proposed_cal → {'gamma': 0.005}

- Finding: every safety margin above 0 was worse. Inflating a single number hurts when the shortage is split proportionally — the cells that over-claim get more

- Same traffic traces, same delayed observations, spare capacity always redistributed, for every controller

- **1. Forecaster: six quantiles (5, 25, 50, 75, 90, 95 %) of each cell's r^* for the next interval**

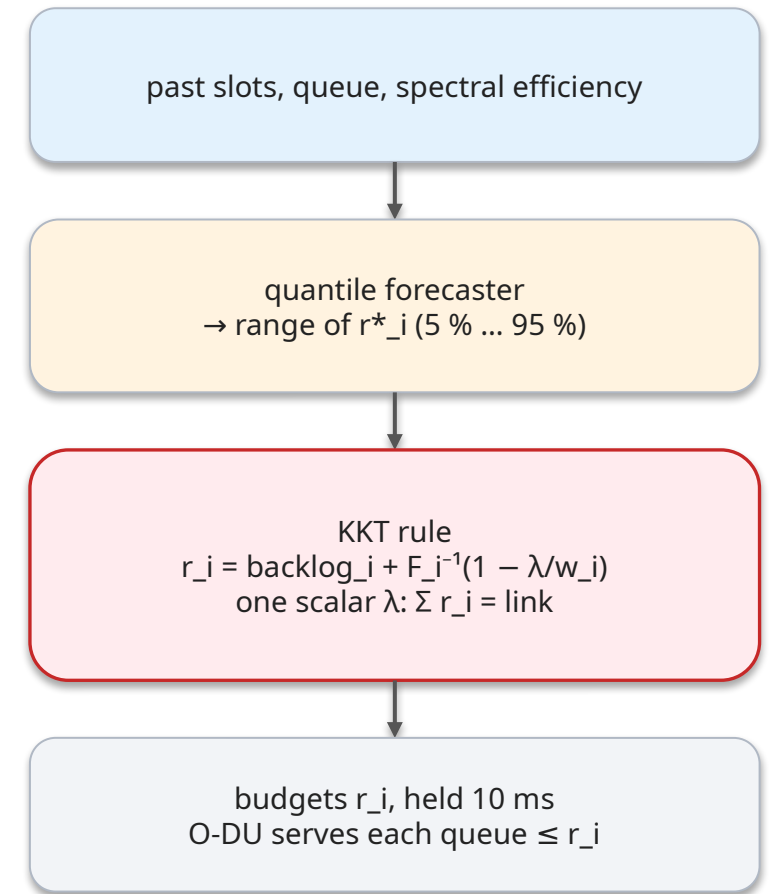
- One gradient-boosted tree model per quantile; 28 features from past slots only; trained on training seeds only
- Targets are divided by the cell's fixed peak, so nothing is fitted on test data

- **2. Allocation rule: maximise the weighted expected served demand under the forecast**

- $\max \sum_i w_i \cdot E[\min(\text{backlog}_i + r_i^*, r_i)]$ subject to $\sum r_i \leq \text{link}$, $0 \leq r_i \leq \text{peak}_i$
- Solution (KKT): $r_i = \text{backlog}_i + F_i^{-1}(1 - \lambda / w_i)$, clipped to $[0, \text{peak}_i]$; λ found by bisection so that $\sum r_i = \text{link}$
- In words: every cell is cut at the same weighted risk of needing one more bit
- $\rightarrow$ wide forecast or high priority $\Rightarrow$ more margin; confident forecast $\Rightarrow \approx$ median; backlog is always funded first

- Optional: an online calibration step (ACI) that shifts a cell's quantile level after misses — tested as an ablation

- Cost: a few microseconds for 8 cells; budgets always feasible (proofs in docs/fhlm/method.md)



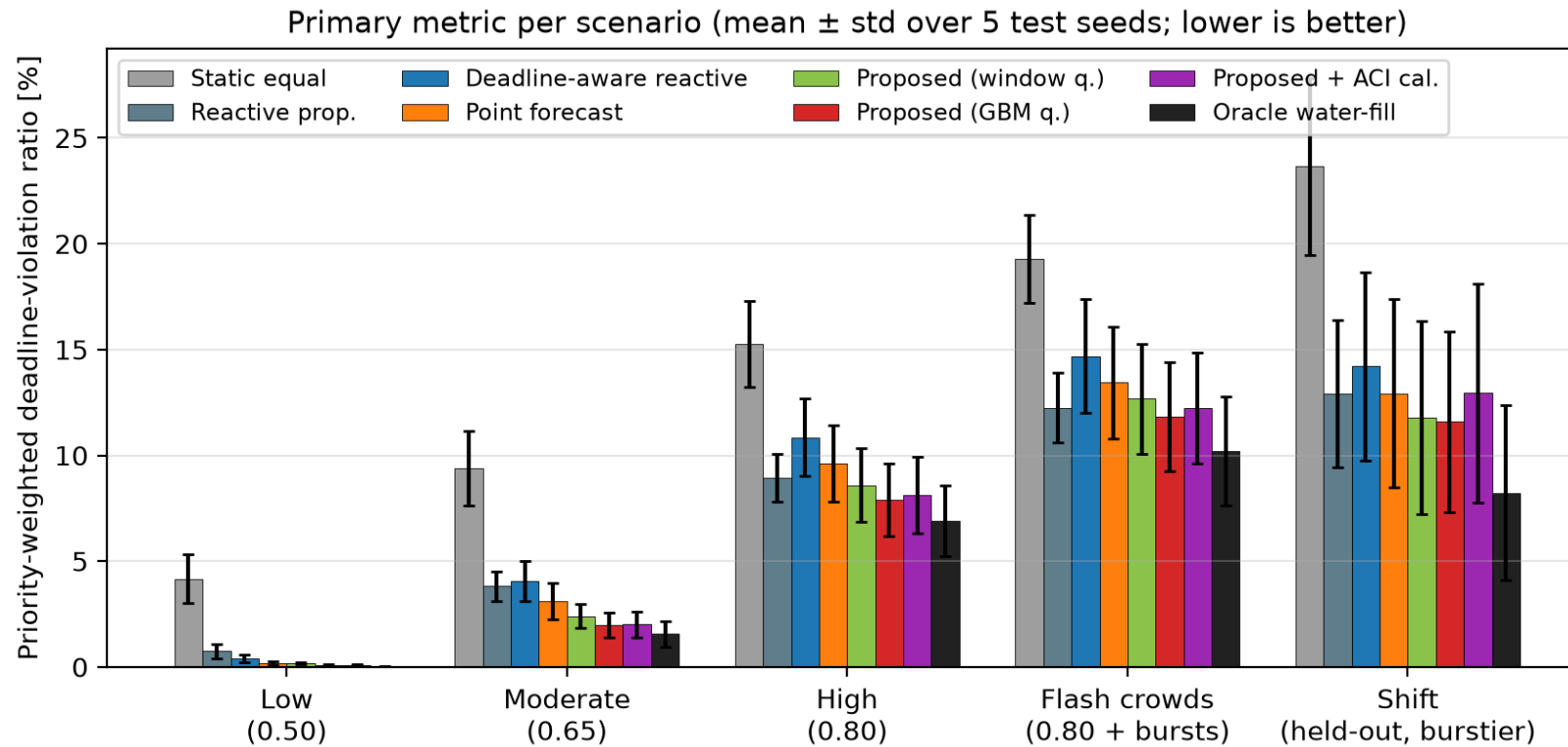
*Borrowed: quantile boosting, ACI, water-filling.
Our part: r^* as the forecast target + the KKT rule over the forecast range under the link constraint, tested against strong baselines.*

Scenario	Mean load / link	Traffic	Used for tuning?
low	0.50	normal	no
moderate	0.65	normal	yes (validation seeds)
high	0.80	normal	yes (validation seeds)
overload_bursts	0.80	flash crowds: load $\times 1.8$ for about 15 % of the time (more than the link can carry)	no
shift (held-out)	0.80	bursts 1.5 $\times$ stronger and 1.5 $\times$ longer, heavier tail — never seen in training or tuning	no

- Traffic (synthetic): smooth Gamma part + heavy-tailed ON/OFF bursts + slow load changes (~ 1 s) + spectral-efficiency drift
- Seeds: training 1000-1015 (loads 0.5-0.95) | validation 2000-2003 | test 3000-3004 — no overlap
- Each run: 20,000 slots = 10 s after a 1,000-slot warm-up; $T = 20$ slots; 8 controllers $\times$ 5 scenarios $\times$ 5 seeds = 200 runs
- Same traffic and same delayed observations for every controller; the forecaster is frozen before testing
- Absolute violation ratios are high on purpose (bursty traffic at 0.8 load); the paired differences between controllers are the result

Results: main score per scenario (5 test seeds, mean $\pm$ std)

Part B — results

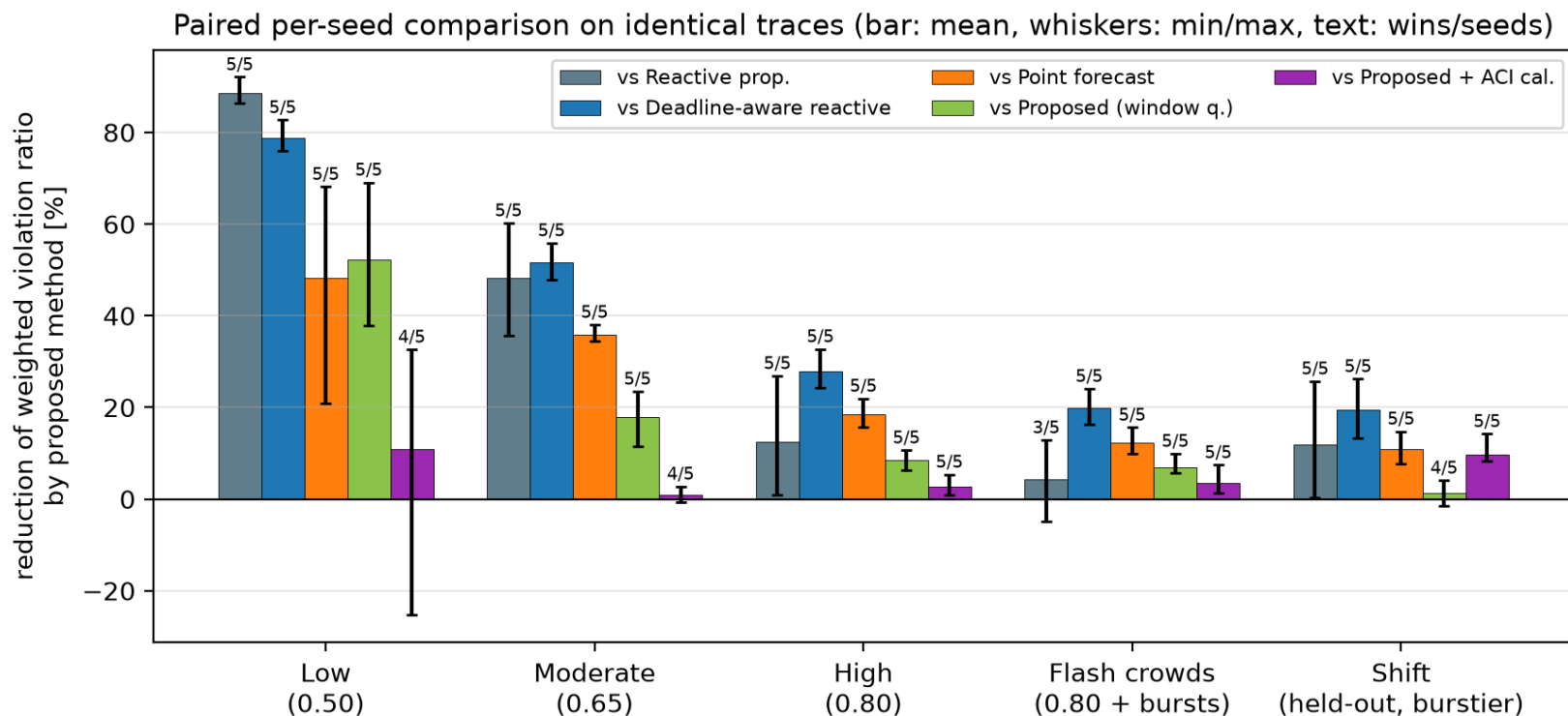


- **Proposed = best real controller in 5 of 5 scenarios**

- vs point forecast (same model, median only): –18 % (high), –10 % (shift)
- vs deadline-aware reactive: –27 % (high), –18 % (shift)
- vs reactive proportional: –12 % (high) but only –3 % under flash crowds (3/5 seeds) → a tie
- Gap to the oracle at high load: 7.9% vs 6.9%
- Error bars are wide because burst timing differs by seed → compare seed by seed (next slide)

Results: seed-by-seed comparison on the same traffic

Part B — results



- **Wins 5/5 seeds vs point forecast and vs deadline-aware reactive in every scenario**

- vs reactive proportional: 5/5 in four scenarios; 3/5 under flash crowds (no claim there)

- **ML ablation (same rule, simple window quantiles instead of the trained model):**

- the model adds 2-51 %; least under the held-out shift (2 %) → the rule, not the model, carries most of the gain

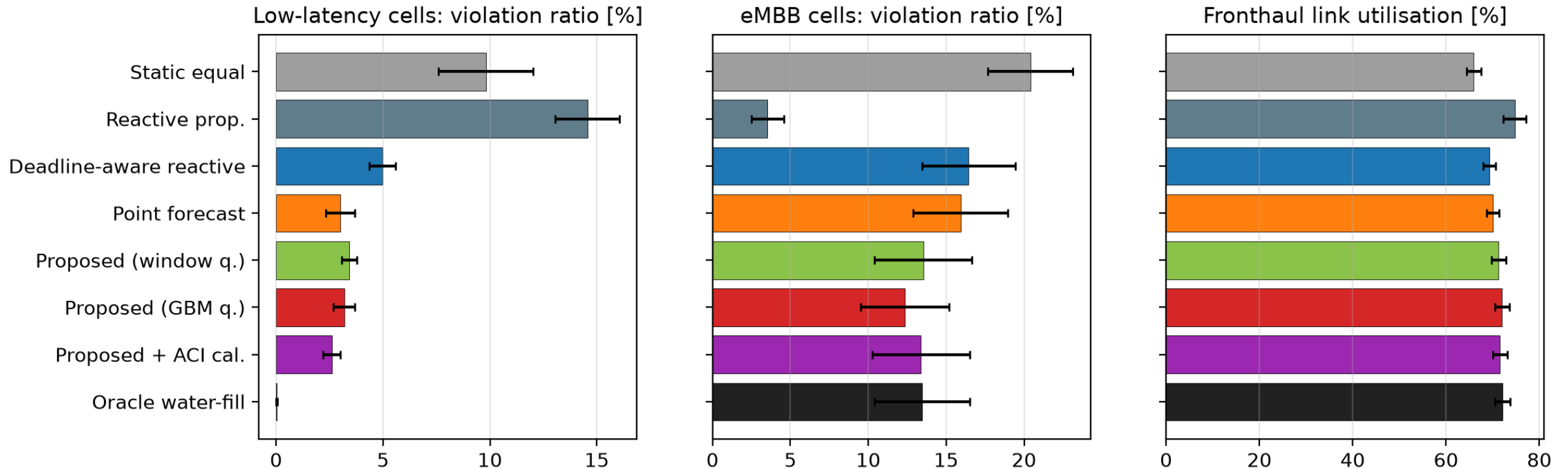
- Online calibration: no gain on normal traffic, harmful under shift (it over-reserves for low-latency cells and starves eMBB) → negative result

- With 5 seeds we do not claim statistical significance; we report win counts and best/worst seed

Where the gain comes from: low-latency vs eMBB (high load)

Part B — results

Scenario 'high': per-class trade-off (mean $\pm$ std over seeds)

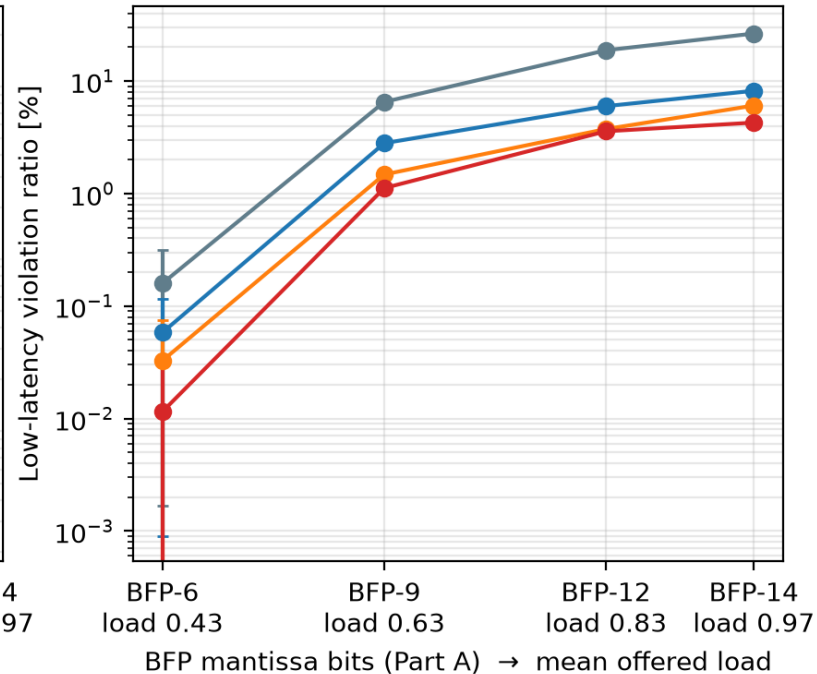
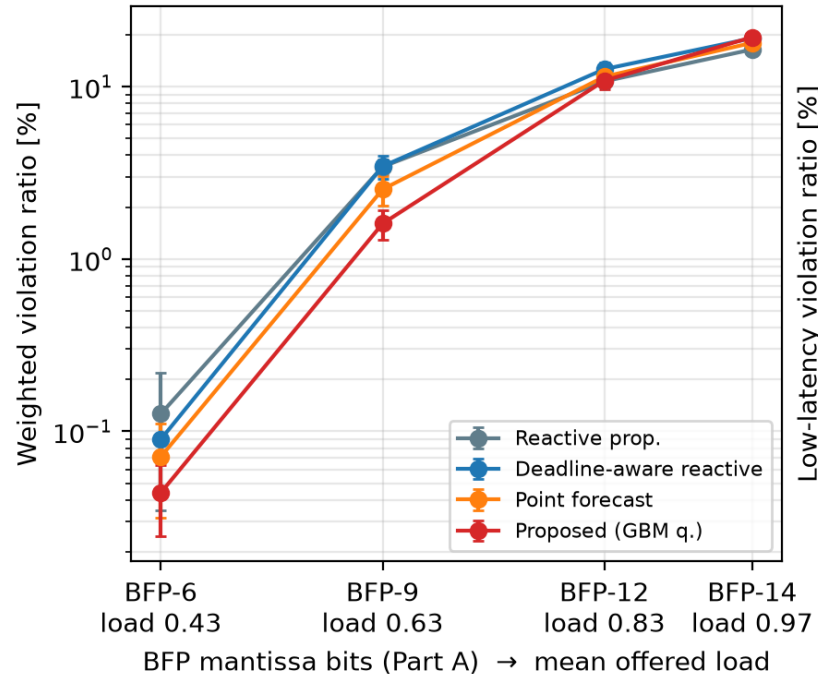


- Reactive proportional: low-latency loss 14.6% vs 3.2% for proposed — controllers that ignore deadlines sacrifice the low-latency cells
- Among deadline-aware controllers, proposed has the lowest eMBB loss (12.3% vs 15.9% for point forecast) and the highest utilisation (72.0%)
- Even the oracle loses 13.4% of eMBB bits at 0.80 load: budgets fixed for 10 ms cannot follow heavy bursts → the control interval matters

Both levers on the same traffic: compression × allocation

A + B

Same user traffic, different compression: compression sets the load, allocation decides the loss



• Same user traffic, 3 seeds, scenario 'moderate'; only the BFP width changes

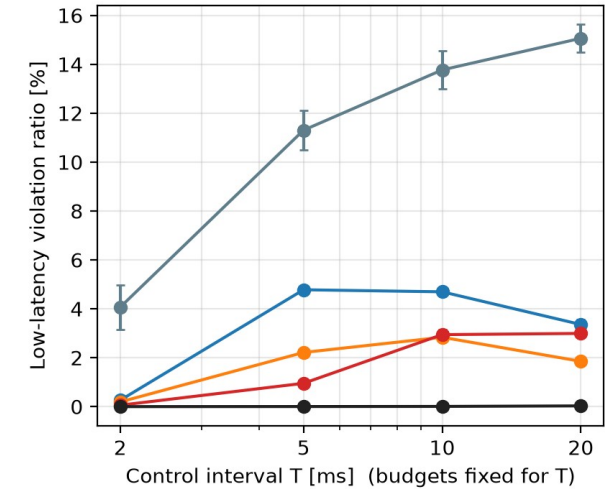
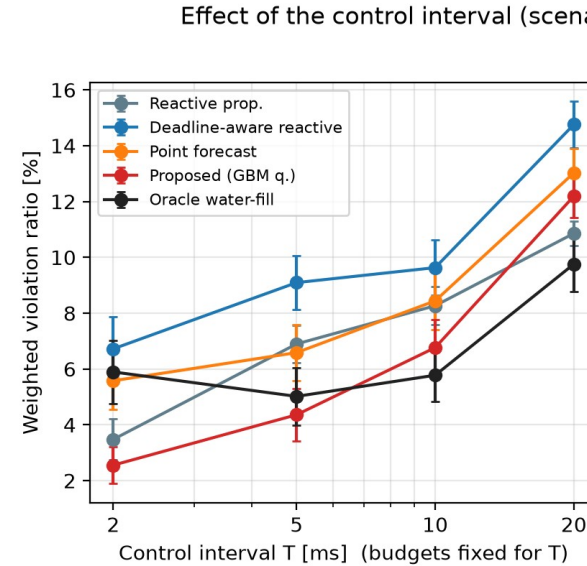
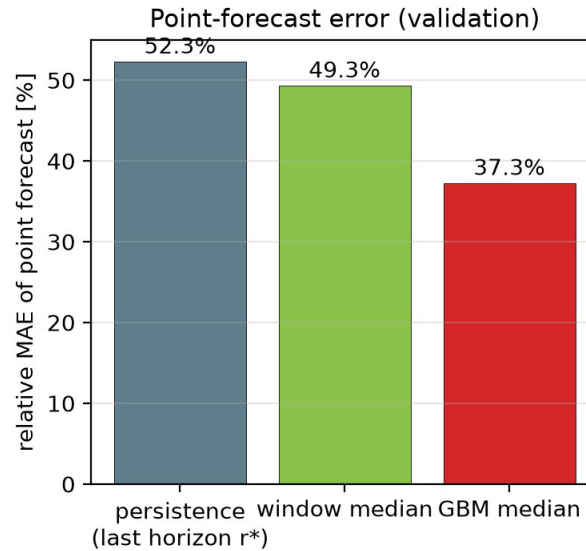
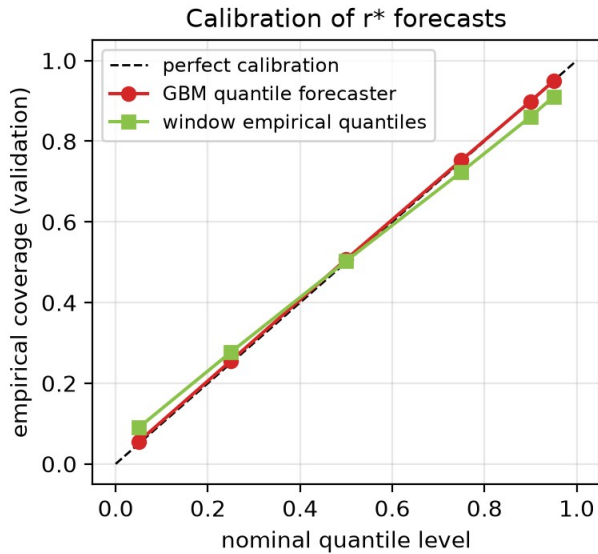
- BFP-6: load 0.43 → proposed 0.04%, reactive prop. 0.13%; best = proposed
- BFP-9: load 0.63 → proposed 1.60%, reactive prop. 3.43%; best = proposed
- BFP-12: load 0.83 → proposed 10.82%, reactive prop. 10.72%; best = reactive prop.
- BFP-14: load 0.97 → proposed 19.34%, reactive prop. 16.42%; best = reactive prop.

• Compression sets the regime; allocation helps most between ~0.5 and ~0.8 load

- At ~1.0 load no allocation rule helps — compress more (Part A) or add capacity
- Not modelled: the signal-quality cost of fewer bits (Part A's NMSE curves cover that side)

Ablations: forecast quality and control interval

Part B — ablation



- Validation: the 5 / 50 / 95 % quantiles are exceeded 5.6% / 50.7% / 94.9% of the time (well calibrated); median error 37% vs 52% for persistence
- **Interval sweep (scenario high, 3 seeds). Reduction by proposed vs deadline-aware reactive / vs reactive proportional / vs oracle water-fill:**
 - T=2 ms: +62% / +27% / +57% | T=5 ms: +52% / +37% / +13% | T=10 ms: +30% / +18% / -17% | T=20 ms: +17% / -12% / -25%
- At T = 2 ms the rule even beats the oracle water-fill (perfect single-number forecast) → the allocation rule, not only the forecast, does the work
- At T = 20 ms (5× the low-latency deadline) reserving the low-latency peak rate wastes capacity and reactive proportional wins → the method is for intervals up to a few times the shortest deadline

- **Part A: compression numbers are quoted from the earlier report and saved figures; the src/ package is not in the repo, so they are not re-run. The matrix-inversion benchmark is re-runnable (needs PyTorch)**
- Part A: benchmark matrices are random and well-conditioned; real channel matrices can be worse
- **Part B: synthetic traffic only — no real radio-unit traces, so no real-world validation is claimed**
- Not a universal win: tie with reactive proportional under flash crowds; loses to it at 20 ms intervals and at ~ 1.0 load; the ML model adds $\approx 2\%$ under the held-out shift
- Online calibration (ACI) did not help — reported as a negative result; the simple-quantile variant is the current fallback
- 5 test seeds $\rightarrow$ we report win counts and best/worst seed, not p-values
- Simplified link: a budget-enforced pipe, no switch queue, one traffic class per cell, no HARQ feedback; 'violation' is a DU queueing deadline, not end-to-end latency
- Decision time ≈ 3.2 ms per 10-ms epoch in Python (reactive baselines ≈ 0.1 ms); a real DU would need a compiled version
- Proofs cover feasibility, equal weighted risk and backlog-first only — no stability or closed-loop optimality claims

Done and checked

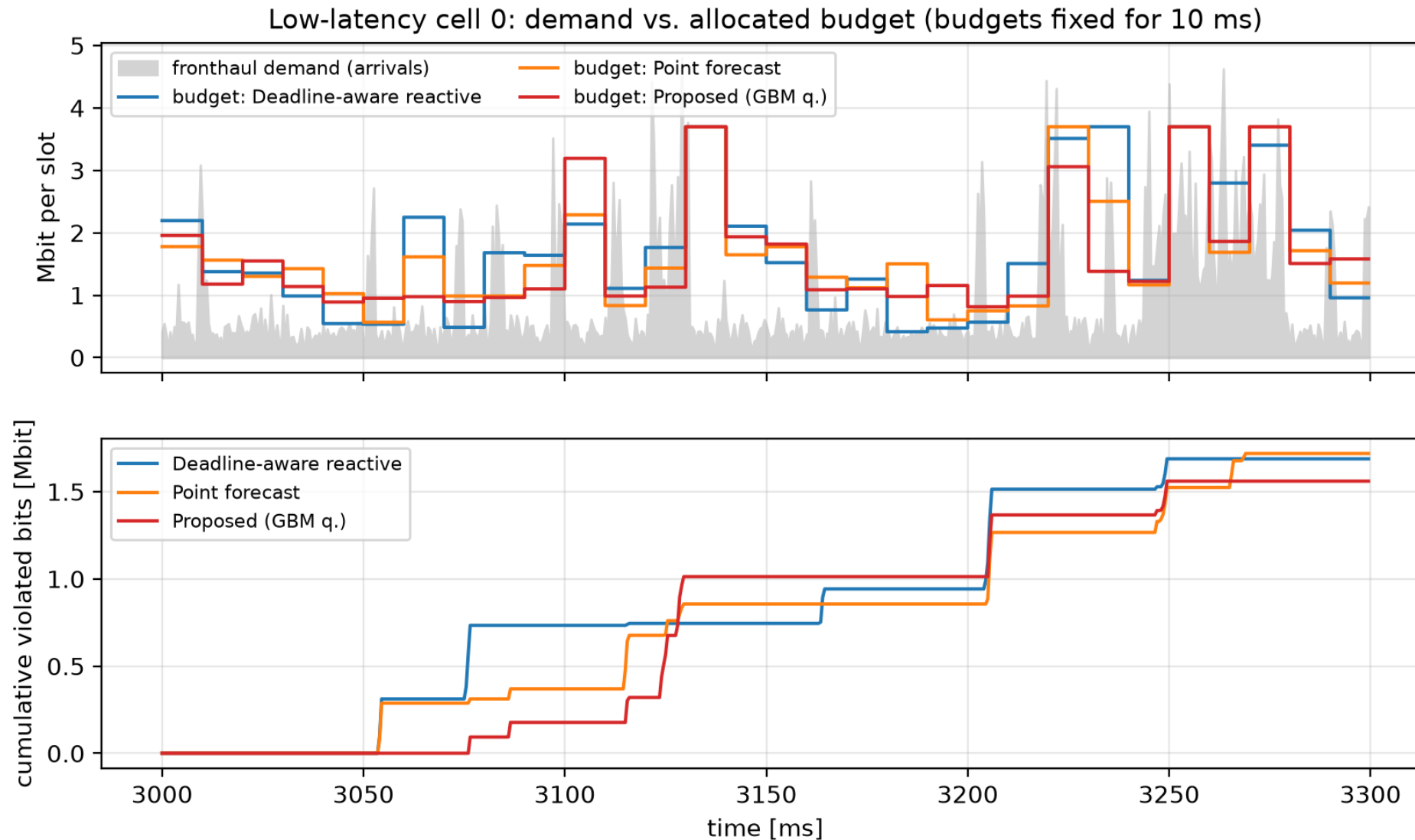
- Part A: compression study (BFP, SVD, RAS-BFP, CSEE, ACAFS) — 4 figure suites + report
- Part A: matrix-inversion benchmark (LU, QR, SVD, Newton-Schulz, learned inverters; $n = 10/100/500$) — CSV + figure
- Part B: slot-level shared-link simulator with deadlines, priorities, telemetry delay (14 tests)
- Part B: 5 reference controllers, tuned on validation seeds; quantile forecaster, calibrated on validation
- Part B: KKT allocation rule with feasibility proof; 200-run comparison, seed-by-seed analysis, interval sweep
- A + B: compression $\times$ allocation experiment; all figures and this deck regenerated by scripts

Planned (not done yet)

- Part A: restore the compression src/ package into the repo and re-run Exp 1-4; benchmark on realistic channel matrices
- A + B: make the compression width a second knob of the allocation rule (trade signal quality for load per cell)
- Part B: real or public traffic traces $\rightarrow$ re-check the forecaster
- Part B: skill-weighted mix of trained and simple quantiles as the fallback (instead of ACI)
- Part B: switch-level packet queue and timing window instead of a budget-enforced pipe; more seeds
- Compiled (C++/numba) decision so the 3 ms Python time fits a real DU

Backup: one trace — budgets vs demand of a low-latency cell

Backup



config.py	units, 7-2x fronthaul constants, cells, link	traffic.py	synthetic arrivals: smooth + bursts + slow changes
demand.py	deadline-feasible rate r^* , backlog rate	simulator.py	slot loop, queues with ages, drops, metrics
controllers.py	5 reference controllers, water-fill	forecast.py	features, quantile boosting, fast tree evaluation
proposed.py	KKT rule + optional calibration	scenarios.py	train / validation / test seeds, 5 scenarios
train.py	train on training seeds, check on validation	run_experiments.py	tune / main / sweep / compression / demo
make_figures.py, make_slides.py	all figures and this deck from CSV	matinv_bench/ (Part A)	matrix-inversion benchmark: classical + learned

- Flow: traffic → simulator ↔ controller (observation 2 slots old → budgets) → metrics / raw JSON → figures → slides
- ML flow: training seeds → features / targets → 6 boosted models (~12 s CPU) → validation check → frozen model → test seeds
- Python 3 with numpy / pandas / scikit-learn / matplotlib; full reproduction ≈ 10 min on a 4-core laptop

Backup: main score [%], mean $\pm$ std over 5 test seeds

Backup

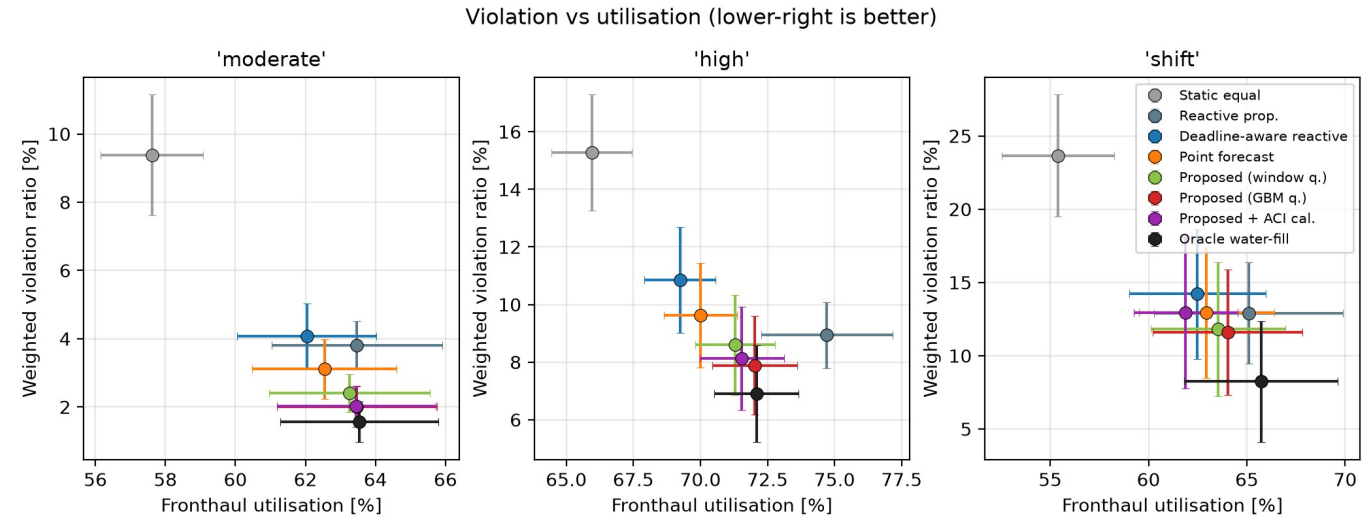
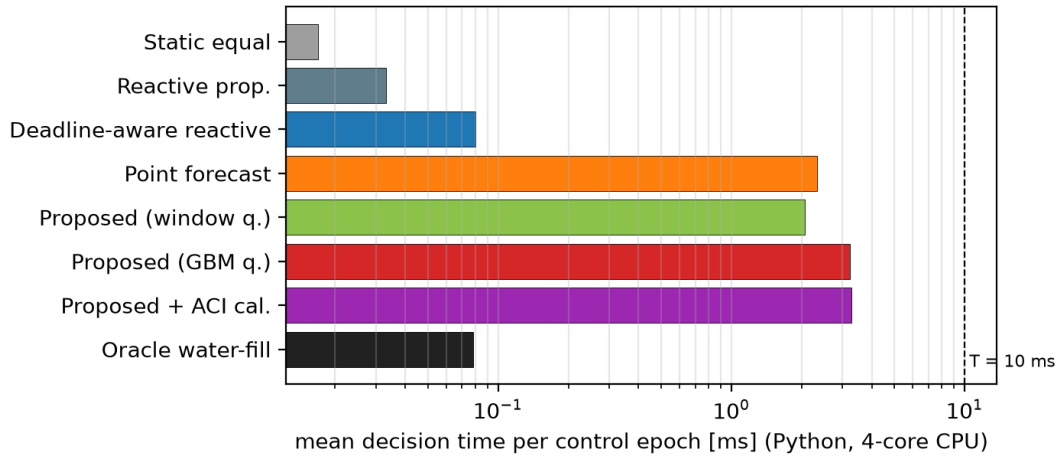
Method	Low 0.50	Moderate 0.65	High 0.80	Flash crowds	Shift (held-out)
Static equal share	4.16 $\pm$ 1.15	9.38 $\pm$ 1.77	15.27 $\pm$ 2.02	19.30 $\pm$ 2.07	23.65 $\pm$ 4.18
Reactive proportional	0.75 $\pm$ 0.34	3.81 $\pm$ 0.71	8.94 $\pm$ 1.15	12.25 $\pm$ 1.65	12.92 $\pm$ 3.47
Deadline-aware reactive	0.39 $\pm$ 0.18	4.07 $\pm$ 0.95	10.86 $\pm$ 1.84	14.70 $\pm$ 2.67	14.21 $\pm$ 4.44
Point forecast + water-fill	0.18 $\pm$ 0.10	3.11 $\pm$ 0.87	9.63 $\pm$ 1.82	13.44 $\pm$ 2.65	12.93 $\pm$ 4.45
Proposed rule, simple quantiles (no ML)	0.17 $\pm$ 0.04	2.41 $\pm$ 0.56	8.60 $\pm$ 1.74	12.67 $\pm$ 2.59	11.80 $\pm$ 4.56
Proposed (GBM quantiles + KKT rule)	0.08 $\pm$ 0.04	2.00 $\pm$ 0.59	7.89 $\pm$ 1.71	11.82 $\pm$ 2.56	11.59 $\pm$ 4.27
Proposed + online calibration	0.09 $\pm$ 0.02	2.02 $\pm$ 0.61	8.12 $\pm$ 1.80	12.24 $\pm$ 2.62	12.95 $\pm$ 5.17
Oracle water-fill (knows the future)	0.01 $\pm$ 0.01	1.56 $\pm$ 0.60	6.90 $\pm$ 1.68	10.21 $\pm$ 2.57	8.23 $\pm$ 4.14

Source: results/fhlm/main_runs.csv (200 runs). Seed-by-seed statistics: results/fhlm/main_paired.csv.

- Objective: $J(r) = \sum_i w_i \cdot E[\min(B_i + X_i, r_i)]$, $X_i \sim$ forecast distribution F_i of r_i^* , B_i = backlog rate
- Derivative: $d/dr_i E[\min(B_i + X_i, r_i)] = P(B_i + X_i > r_i) = 1 - F_i(r_i - B_i) \rightarrow J$ is concave and separable
- Lagrangian with the link constraint (λ) and the box constraints (μ_i, v_i)
- Stationarity for an interior r_i : $w_i (1 - F_i(r_i - B_i)) = \lambda \Rightarrow r_i = B_i + F_i^{-1}(1 - \lambda / w_i)$
- Clip to $[0, \text{peak}_i]$; $\sum_i r_i(\lambda)$ decreases in $\lambda \Rightarrow$ one λ^* by bisection (60 steps)
- Property 1 (feasibility): budgets always respect the caps and $\sum r_i \leq \text{link}$ — checked by `test_kkt_allocation_properties`
- Property 2 (equal weighted risk): at the optimum every interior cell has $w_i \cdot P(\text{needs one more bit}) = \lambda^*$
- Property 3 (backlog first): $F_i(x) = 0$ for $x < 0 \Rightarrow$ the marginal value below B_i is $w_i \Rightarrow$ backlog is funded before any uncertain demand of a cell with $w \leq \lambda^*$
- A single-number forecast (F_i a step) gives a priority-ordered fill = the point-forecast water-fill baseline
- F_i^{-1} : straight lines through the 6 forecast quantiles, extended linearly above 95 %; quantiles sorted to keep them monotone

Backup: decision time and violation-vs-utilisation trade-off

Backup



Decision times are Python on a 4-core CPU inside the simulator (feature construction + 1,800 trees with a vectorised traversal + bisection). The rule itself is < 0.1 ms.

- O-RAN WG4, Control, User and Synchronization Plane Specification (O-RAN.WG4.CUS.0) [standard] — 7-2x, BFP compression (Annex A.1), T2a windows. 3GPP TR 38.801 [standard]; 3GPP TR 38.901 (TDL-A channel) [standard]; IEEE 802.1CM-2018 [standard].
- L. M. P. Larsen, A. Checko, H. L. Christiansen, 'A Survey of the Functional Splits Proposed for 5G Mobile Crosshaul Networks', IEEE COMST 21(1), 2019. DOI 10.1109/COMST.2018.2868805 [peer-reviewed]
- G. O. Pérez, J. A. Hernández, D. Larrabeiti, 'Fronthaul Network Modeling and Dimensioning Meeting Ultra-Low Latency Requirements for 5G', JOCN 10(6), 2018. DOI 10.1364/JOCN.10.000573; IEEE Access 2019, DOI 10.1109/ACCESS.2019.2923020 [peer-reviewed]
- L. Wang, S. Zhou, 'On the Fronthaul Statistical Multiplexing Gain', IEEE Commun. Lett. 21(5), 2017. DOI 10.1109/LCOMM.2017.2653120 [peer-reviewed]
- S. Lagén, X. Gelabert, A. Hansson, M. Requena, L. Giupponi, 'Fronthaul Compression Control for Shared Fronthaul Access Networks', IEEE Commun. Mag. 2022. DOI 10.1109/MCOM.001.2100959 [peer-reviewed]
- D. Bega, M. Gramaglia, M. Fiore, A. Banchs, X. Costa-Pérez, 'DeepCog', IEEE INFOCOM 2019, DOI 10.1109/INFOCOM.2019.8737488; 'AZTEC', IEEE INFOCOM 2020, DOI 10.1109/INFOCOM41043.2020.9155299 [peer-reviewed]
- F. Kavehmadavani, V.-D. Nguyen, T. X. Vu, S. Chatzinotas, 'Intelligent Traffic Steering in Beyond 5G Open RAN Based on LSTM Traffic Prediction', IEEE TWC 2023. DOI 10.1109/TWC.2023.3254903 [peer-reviewed]
- K. M. Cohen, S. Park, O. Simeone, P. Popovski, S. Shamai, 'Guaranteed Dynamic Scheduling of URLLC Traffic via Conformal Prediction', IEEE WCL 2023 [peer-reviewed]; I. Gibbs, E. Candès, 'Adaptive Conformal Inference Under Distribution Shift', NeurIPS 2021 [peer-reviewed]
- V. Kasuluru, L. Blanco, C. J. Vaca-Rubio, E. Zeydan, 'On the Impact of PRB Load Uncertainty Forecasting for Sustainable Open RAN', arXiv:2407.14400, 2024 [preprint]
- Eckart & Young 1936 (best low-rank approximation = truncated SVD); Halko, Martinsson, Tropp, SIAM Review 2011 (randomised sketching for low-rank factorisation); Willinger et al., IEEE/ACM ToN 1997 (heavy-tailed ON/OFF sources); Hadley & Whitin 1963 (constrained newsvendor); Le Boudec & Thiran, Network Calculus, 2001.